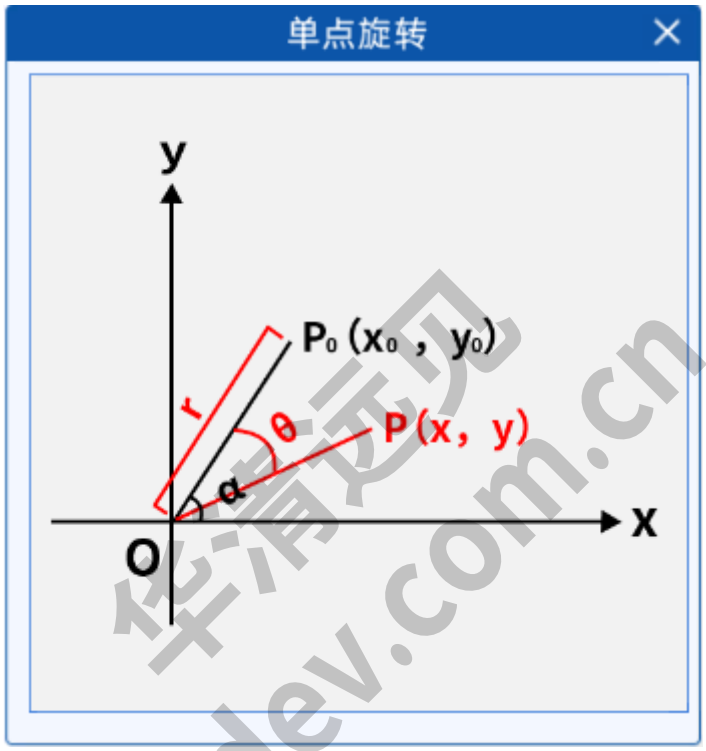
华清远见 HQYJ.COM

图像旋转

图像旋转是指图像以某一点为旋转中心，将图像中的所有像素点都围绕该点旋转一定的角度，并且旋转后的像素点组成的图像与原图像相同。

1. 单点旋转

首先我们以最简单的一个点的旋转为例子，且以最简单的情况举例，令旋转中心为坐标系中心O（0，0），假设有一点$P_0(x_0, y_0)$，P_0离旋转中心O的距离为r，OP_0与坐标轴x轴的夹角为α，P_0绕O顺时针旋转θ角后对应的点为$P(x, y)$，如下图所示：



华清远见 fsdev.com.cn

那么我们可以得到如下关系：

$$x_0 = r \times \cos\alpha$$

$$y_0 = r \times \sin\alpha$$

$$x = r \times \cos(\alpha - \theta) = r\cos\alpha\cos\theta + r\sin\alpha\sin\theta = x_0\cos\theta + y_0\sin\theta$$

$$y = r \times \sin(\alpha - \theta) = r\sin\alpha\cos\theta - r\cos\alpha\sin\theta = -x_0\sin\theta + y_0\cos\theta$$

用矩阵来表示就是

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} * \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}$$

然而，在OpenCV中，旋转时是以图像的左上角为旋转中心，且以逆时针为正方向，因此上面的例子中其实是个负值，那么该矩阵可写为：

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} * \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}$$

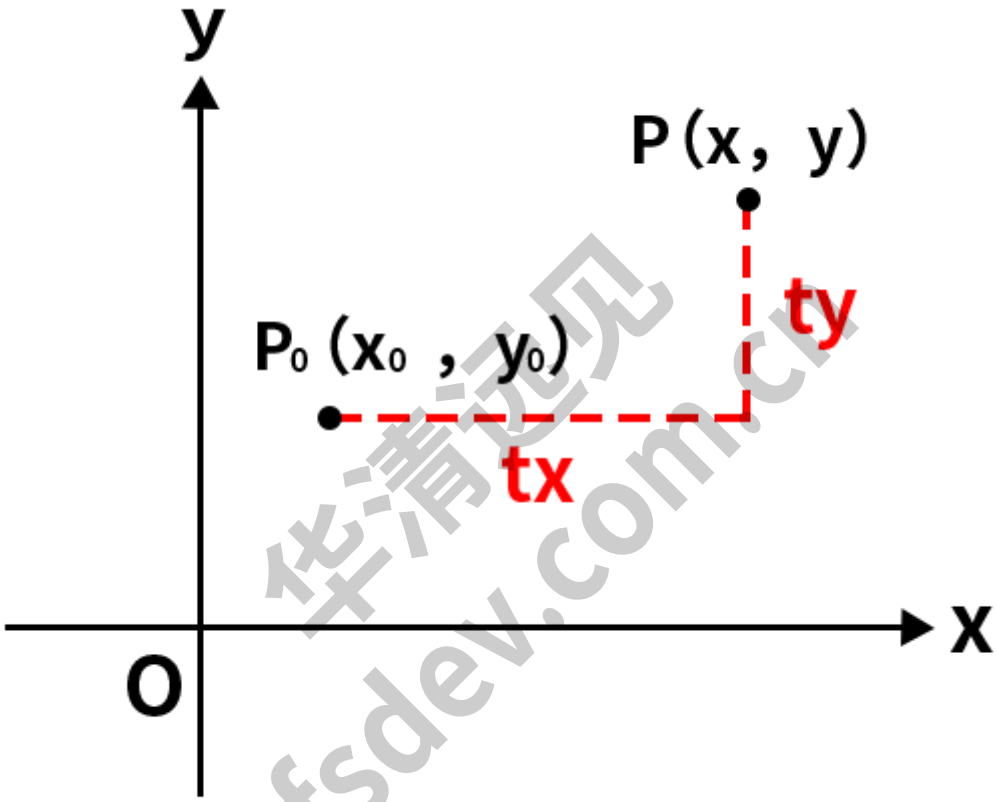
其中，

$$\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

也被称作旋转矩阵。然而我们所要的不仅仅是可以围绕图像左上角进行旋转，而是可以围绕任意点进行旋转。那么我们可以将其转化成绕原点的旋转，其过程为：

1. 首先将旋转点移到原点
2. 按照上面的旋转矩阵进行旋转得到新的坐标点
3. 再将得到的旋转点移回原来的位置

也就是说，在以任意点为旋转中心时，除了要进行旋转之外，还要进行平移操作。那么当点经过平移后得到P点时，如下图所示：



那么我们就可以得到：

$$x = x_0 + t_x$$

$$y = y_0 + t_y$$

写成矩阵的形式为：

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x_0 \\ y_0 \\ 1 \end{bmatrix}$$

于是

$$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

华清远见 HQYJ.COM

也被叫做平移矩阵，相反的，从P移到点时，其平移矩阵为：

$$\begin{bmatrix} 1 & 0 & -t_x \\ 0 & 1 & -t_y \\ 0 & 0 & 1 \end{bmatrix}$$

我们将原始的旋转矩阵也扩展到3*3的形式：

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x_0 \\ y_0 \\ 1 \end{bmatrix}$$

从平移和旋转的矩阵可以看出，3x3矩阵的前2x2部分是和旋转相关的，第三列与平移相关。有了上面的表达式之后，我们就可以得到二维空间中绕任意点旋转的旋转矩阵了，只需要将旋转矩阵先左乘

$$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

，再右乘

$$\begin{bmatrix} 1 & 0 & -t_x \\ 0 & 1 & -t_y \\ 0 & 0 & 1 \end{bmatrix}$$

即可得到最终的矩阵，其结果为：

$$M = \begin{bmatrix} \cos\theta & -\sin\theta & (1-\cos\theta)t_x + t_y * \sin\theta \\ \sin\theta & \cos\theta & (1-\cos\theta)t_y + t_x * \sin\theta \\ 0 & 0 & 1 \end{bmatrix}$$

于是我们就可以根据这个矩阵计算出图像中任意一点绕某点旋转后的坐标了，这个矩阵学名叫做仿射变换矩阵，而仿射变换是一种二维坐标到二维坐标之间的线性变换，也就是只涉及一个平面内二维图形的线性变换，图像旋转就是仿射变换的一种。它保持了二维图形的两种性质：

1. 平直性：直线经过变换后依然是直线。
2. 平行性：平行线经过变换后依然是平行线。

2. 图片旋转

明白了单个点的旋转过程之后，其实图像旋转也很好理解，就是将图像里的每个像素点都带入仿射变换矩阵里，从而得到旋转后的新坐标。在OpenCV中，要得到仿射变换矩阵可以使用cv2.getRotationMatrix2D()，通过这个函数即可直接获取到上面的旋转矩阵，该函数需要接收的参数为：

- Center：表示旋转的中心点，是一个二维的坐标点(x,y)
- Angle：表示旋转的角度
- Scale：表示缩放比例，可以通过该参数调整图像相对于原始图像的大小变化

因此，在本实验中只需要在组件中填好图片要旋转的角度与缩放的比例即可。

华清远见 fsdev.com.cn

华清远见 HQYJ.COM

但是这里会有一个问题，由于三角函数的值是小数，那么其乘积也会是小数，虽然OpenCV中会对其进行取整操作，但是像素点旋转之后的取整结果也有可能重合，这样就会导致可能会在旋转的过程中丢失一部分原始的像素信息。并且如果使用了scale参数进行图像的缩放的话，当图像放大时，比如一个10*10的图像放大成20*20，图像由100个像素点变成400个像素点，那么多余的300个像素点是怎么来的？而当图像缩小时，比如一个20*20的图像缩小为10*10的图像，需要丢掉300个像素点，那到底要怎么丢才能保证图像还能是一个正常的图像？因此我们需要一种方法来帮我们计算旋转后的图像中每一个像素点所对应的像素值，从而保证图像的完整性，这种方法就叫做插值法。

3. 插值方法

在图像处理和计算机图形学中，插值（Interpolation）是一种通过已知数据点之间的推断或估计来获取新数据点的方法。它在图像处理中常用于处理图像的放大、缩小、旋转、变形等操作，以及处理图像中的像素值。

图像插值算法是为了解决图像缩放或者旋转等操作时，由于像素之间的间隔不一致而导致的信息丢失和图像质量下降的问题。当我们对图像进行缩放或旋转等操作时，需要在新的像素位置上计算出对应的像素值，而插值算法的作用就是根据已知的像素值来推测未知位置的像素值。本实验提供了五种常见的插值算法，下面一一介绍。

3.1 最近邻插值

首先给出目标点与原图像点之间坐标的计算公式：

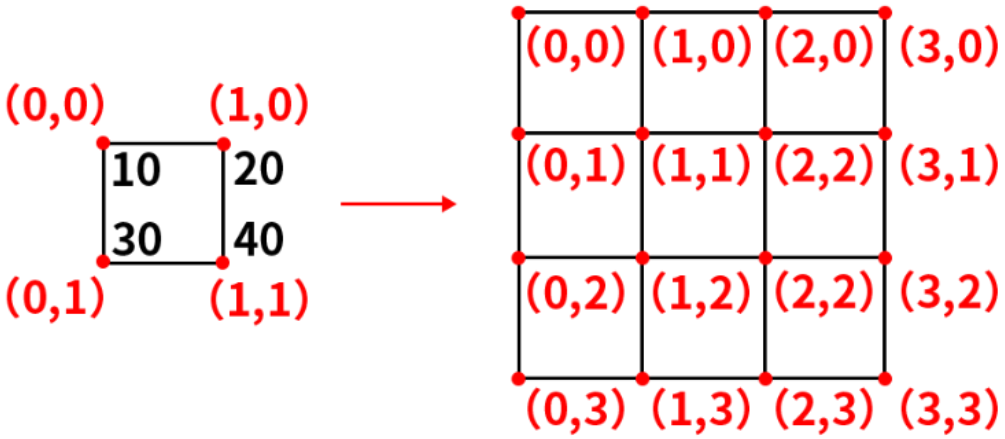
$$srcX = dstX * \frac{srcWidth}{dstWidth}$$

$$srcY = dstY * \frac{srcHeight}{dstHeight}$$

其中，dstX表示目标图像中某点的x坐标，srcWidth表示原图的宽度，dstWidth表示目标图像的宽度；dstY表示目标图像中某点的y坐标，srcHeight表示原图的高度，dstHeight表示目标图像的高度。而srcX和srcY则表示目标图像中的某点对应的原图中的点的x和y的坐标。通俗的讲，该公式就是让目标图像中的每个像素值都能找到对应的原图中的像素值，这样才能根据不同的插值方法来获取新的像素值。

根据该公式，我们就可以得到每一个目标点所对应的原图像的点，比如一个2*2的图像放大到4*4，如下图所示，其中红色的为每个像素点的坐标，绿色的则表示该像素点的像素值。

华清远见 fsdev.com.cn



那么根据公式我们就可以计算出放大后的图像（0，0）点对应的原图像中的坐标为：

$$srcx = 0 * (\frac{2}{4}) = 0$$

$$srcy = 0 * (\frac{2}{4}) = 0$$

也就是原图中的（0，0）点，而最近邻插值的原则是：目标像素点的像素值与经过该公式计算出来的对应的像素点的像素值相同，如出现小数部分需要进行取整。那么放大后图像的（0，0）坐标处的像素值就是原图像中（0，0）坐标处的像素值，也就是10。

接下来就是计算放大后图像（1，0）点对应的原图像的坐标，还是带入公式：

$$srcy = 1 * (\frac{2}{4}) = 0.5$$

$$srcy = 0 * (\frac{2}{4}) = 0$$

也就是原图中的（0.5，0）点，因此需要对计算出来的坐标值进行取整，取整后的结果为（0，0），也就是说放大后的图像中的（1，0）坐标处对应的像素值就是原图中（0，0）坐标处的像素值，其他像素点计算规则与此相同。

3.2 双线性插值

双线性插值算法是一种比较好的图像缩放算法，它充分的利用了原图中虚拟点四周的四个真实存在像素点的值来共同决定目标图中的一个像素点的值，因此缩放效果比简单的最邻近插值要好很多，缩放后图像质量高，不会出现值不连续的情况。

那我们先说一下线性插值是什么，假如已知两个点(x_0, y_0)和(x_1, y_1)，我们要计算$[x_0, y_0]$区间内某一位置x在直线上的y值，那么计算过程为：

$$\frac{y - y_0}{x - x_0} = \frac{y_1 - y_0}{x_1 - x_0}$$

$$y = \frac{x_1 - x}{x_1 - x_0} y_0 + \frac{x - x_0}{x_1 - x_0} y_1$$

仔细看公式，其实就是计算距离，并将距离作为一个权重用于y_0和y_1的加权求和。这就是线性插值，而双线性插值本质上就是在两个方向上做线性插值。

还是给出目标点与原图像中点的计算公式：

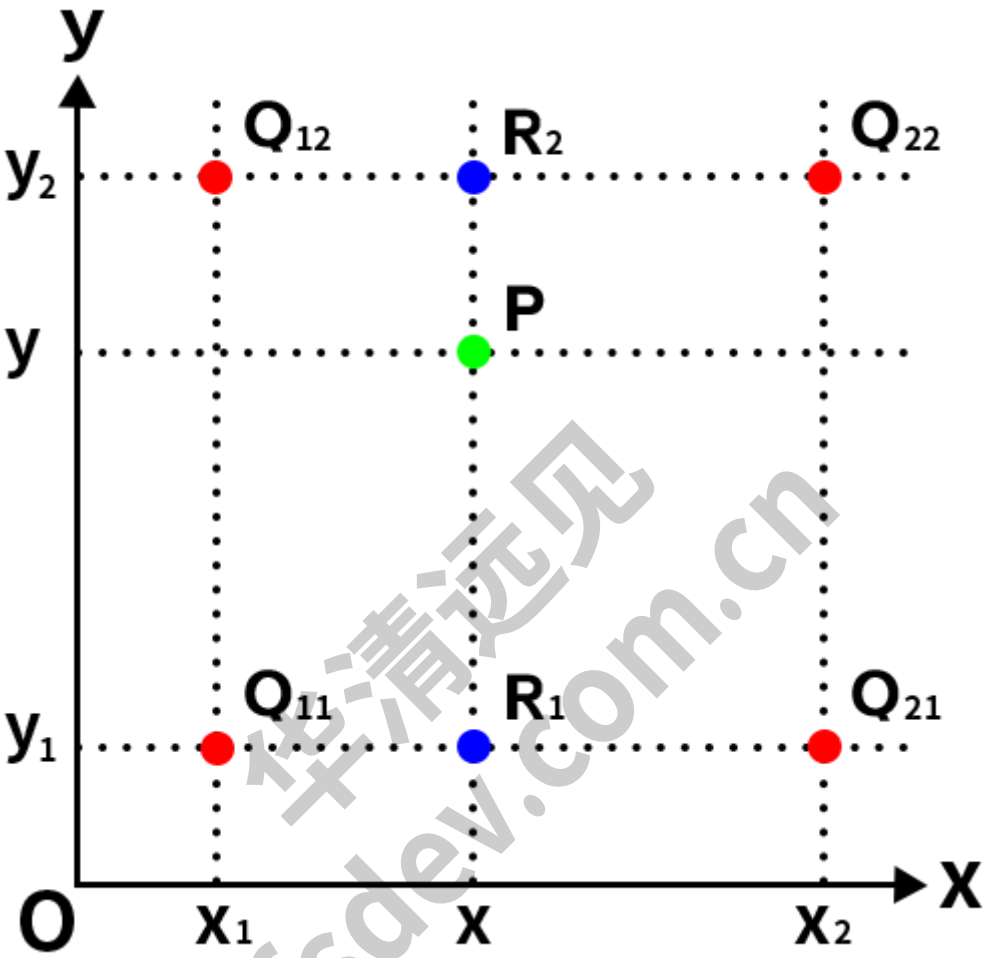
华清远见 fsdev.com.cn

华清远见 HQYJ.COM

$$srcX = dstX * \frac{srcWidth}{dstWidth}$$

$$srcY = dstY * \frac{srcHeight}{dstHeight}$$

比如我们根据上述公式计算出了新图像中的某点所对应的原图像的点P，其周围的点分别为Q12、Q22、Q11、Q21， 要插值的P点不在其周围点的连线上，这时候就需要用到双线性插值了。首先延申P点得到P和Q11、Q21的交点R1与P和Q12、Q22的交点R2，如下图所示：



华清远见 fsdev.com.cn

然后根据Q11、Q21得到R1的插值，根据Q12、Q22得到R2的插值，然后根据R1、R2得到P的插值即可，这就是双线性插值。以下是计算过程：

首先计算R1和R2的插值：

$$f(R_1) \approx \frac{x_2 - x}{x_2 - x_1} f(Q_{11}) + \frac{x - x_1}{x_2 - x_1} f(Q_{21})$$

$$f(R_2) \approx \frac{x_2 - x}{x_2 - x_1} f(Q_{12}) + \frac{x - x_1}{x_2 - x_1} f(Q_{22})$$

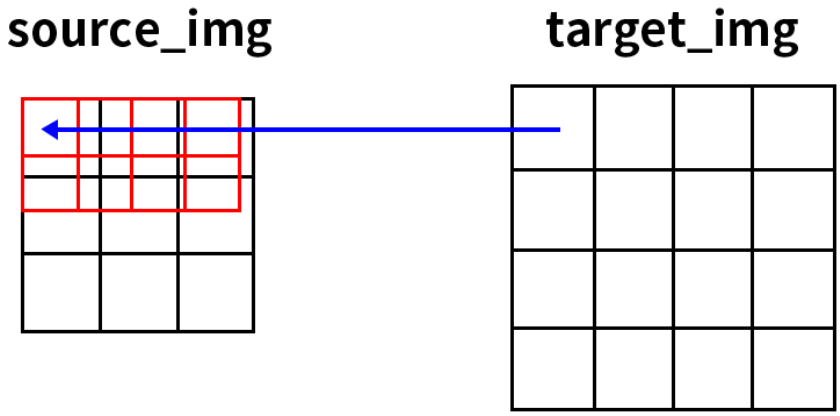
然后根据R1和R2计算P的插值：

$$f(P) \approx \frac{y_2 - y}{y_2 - y_1} f(R_1) + \frac{y - y_1}{y_2 - y_1} f(R_2)$$

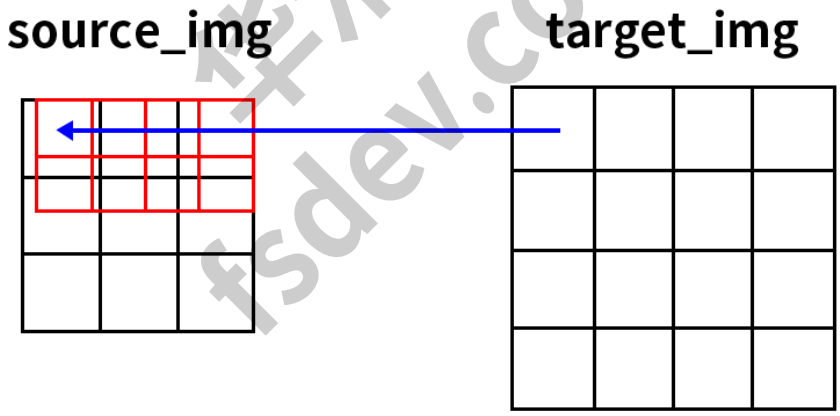
这样就得到了P点的插值。注意此处如果先在y方向插值、再在x方向插值，其结果与按照上述顺序双线性插值的结果是一样的。

华清远见 HQYJ.COM

双线性插值的对应关系看似比较清晰，但还是有2个问题。首先是根据坐标系的不同，产生的结果不同，这张图是左上角为坐标系原点的情况，我们可以发现最左边x=0的点都会有概率直接复制到目标图像中（至少原点肯定是这样），而且就算不和原图像中的点重合，也相当于进行了1次单线性插值（带入到权重公式中会发现结果）。

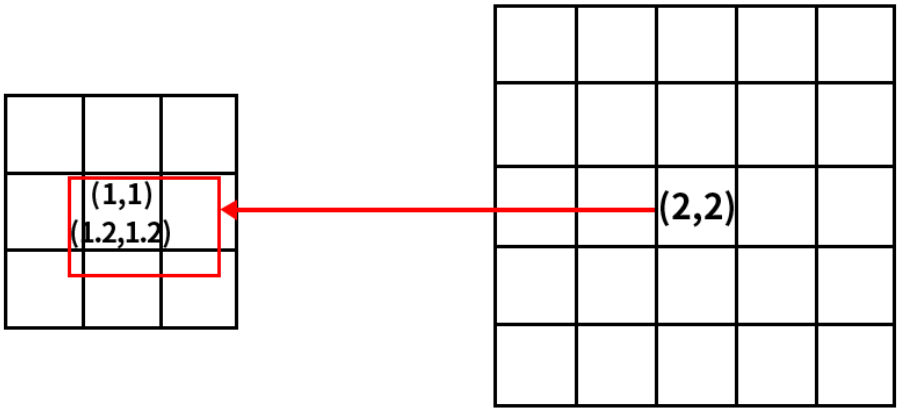


下面这张图是右上角为坐标系原点的情况，我们可以发现最右面的点都会有概率直接复制到目标图像中（至少原点肯定是这样），而且就算不和原图像中的点重合，也相当于进行了1次单线性插值。那么当我们采用不同的坐标系时产生的结果是不一样的，而且无论我们采用什么坐标系，最左侧和最右侧（最上侧和最下侧）的点是“不公平的”，这是第一个问题。



第二个问题时整体的图像相对位置会发生变化。如下图所示，左侧是原图像(3，3)，右侧是目标图像(5，5)，原图像的几何中心点是(1,1)，目标图像的几何中心点是(2,2)，根据对应关系，目标图像的几何中心点对应的原图像的位置是(1.2,1.2)，那么问题来了，目标图像的原点(0,0)和原始图像的原点是重合的，但是目标图像的几何中心点相对于原始图像的几何中心点偏右下，那么整体图像的位置会发生偏移，所以参与计算的点相对都往右下偏移会产生相对的位置信息损失。这是第二个问题。

华清远见 fsdev.com.cn



因此，在OpenCV中，为了解决这两个问题，将公式进行了优化，如下所示：

$$srcX = (dstX + 0.5) * \frac{srcWidth}{dstWidth} - 0.5$$

$$srcY = (dstY + 0.5) * \frac{srcHeight}{dstHeight} - 0.5$$

使用该公式计算出原图中的对应坐标后再进行插值计算，就不会出现上面的情况了。

3.3 像素区域插值

像素区域插值主要分两种情况，缩小图像和放大图像的工作原理并不相同。

当使用像素区域插值方法进行缩小图像时，它就会变成一个均值滤波器（滤波器其实就是一个核，这里只做简单了解，后面实验中会介绍），其工作原理可以理解为对一个区域内的像素值取平均值。

当使用像素区域插值方法进行放大图像时，如果图像放大的比例是整数倍，那么其工作原理与最近邻插值类似；如果放大的比例不是整数倍，那么就会调用双线性插值进行放大。

其中目标像素点与原图像的像素点的对应公式如下所示：

$$srcX = dstX * \frac{srcWidth}{dstWidth}$$

$$srcY = dstY * \frac{srcHeight}{dstHeight}$$

3.4 双三次插值

与双线性插值法相同，该方法也是通过映射，在映射点的邻域内通过加权来得到放大图像中的像素值。不同的是，双三次插值法需要原图像中近邻的16个点来加权。

目标像素点与原图像的像素点的对应公式如下所示：

$$srcX = dstX * \frac{srcWidth}{dstWidth}$$

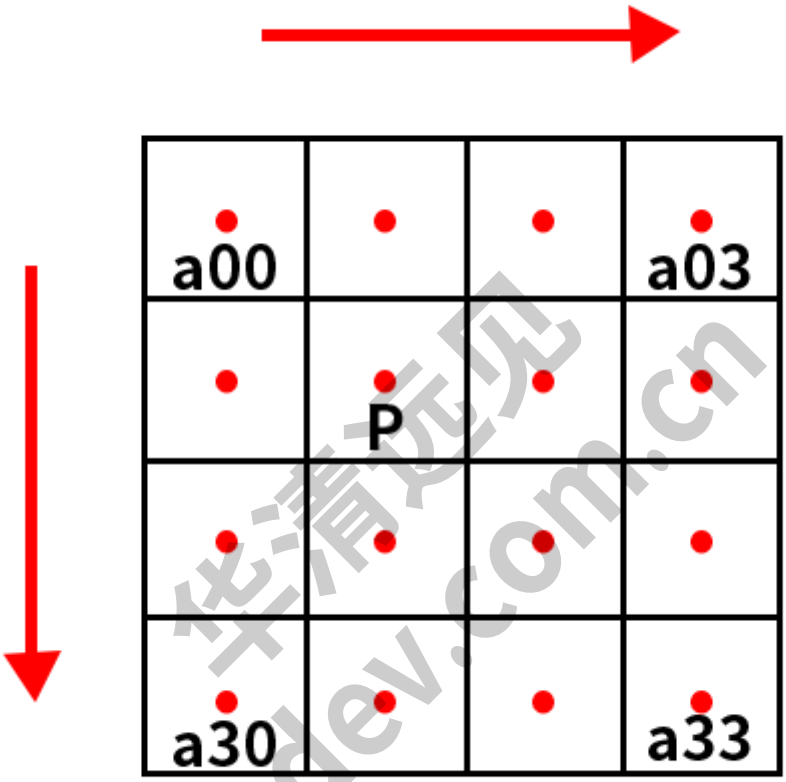
$$srcY = dstY * \frac{srcHeight}{dstHeight}$$

华清远见 fsdev.com.cn

华清远见 HQYJ.COM

下面我们举例说明，假设原图像A大小为m*n，缩放后的目标图像B的大小为M*N。其中A的每一个像素点是已知的，B是未知的，我们想要求出目标图像B中每一个像素点（X,Y）的值，必须先找出像素（X,Y）在原图像A中对应的像素（x,y），再根据原图像A距离像素（x,y）最近的16个像素点作为计算目标图像B（X,Y）处像素值的参数，利用BiCubic基函数求出16个像素点的权重，图B像素（x,y）的值就等于16个像素点的加权叠加。

假如下图中的P点就是目标图像B在（X,Y）处根据上述公式计算出的对应于原图像A中的位置，P的坐标位置会出现小数部分，所以我们假设P点的坐标为（x+u,y+v），其中x、y表示整数部分，u、v表示小数部分，那么我们就可以得到其周围的最近的16个像素的位置，我们用a（i，j）（i，j=0,1,2,3）来表示，如下图所示。



华清远见 fsdev.com.cn

然后给出BiCubic函数：

$$W(x)=\begin{cases}(a+2)|x|^3-(a+3)|x|^2+1 & |x|\le 1\\ a|x|^3-5a|x|^2+8a|x|-4a & 1<|x|<2\\ 0 & 其它\end{cases}$$

其中，a一般取-0.5或-0.75。

我们要做的就是将上面的16个点的坐标带入函数中，获取16像素所对应的权重$W(x)$。然而BiCubic函数是一维的，所以我们需要将像素点的行与列分开计算，比如a00这个点，我们需要将x=0带入BiCubic函数中，计算a00点对于P点的x方向的权重，然后将y=0带入BiCubic函数中，计算a00点对于P点的y方向的权重，其他像素点也是这样的计算过程，最终我们就可以得到P所对应的目标图像B在（X,Y）处的像素值为：

$$B(X,Y)=\sum_{i=0}^{3}\sum_{j=0}^{3}a_{ij}\times W_{(i)}\times W_{(j)}$$

依此办法我们就可以得到目标图像中所有的像素点的像素值。

3.5 Lanczos插值

Lanczos插值方法与双三次插值的思想是一样的，不同的就是其需要的原图像周围的像素点的范围变成了8*8，并且不再使用BiCubic函数来计算权重，而是换了一个公式计算权重。

首先还是目标像素点与原图像的像素点的对应公式如下所示：

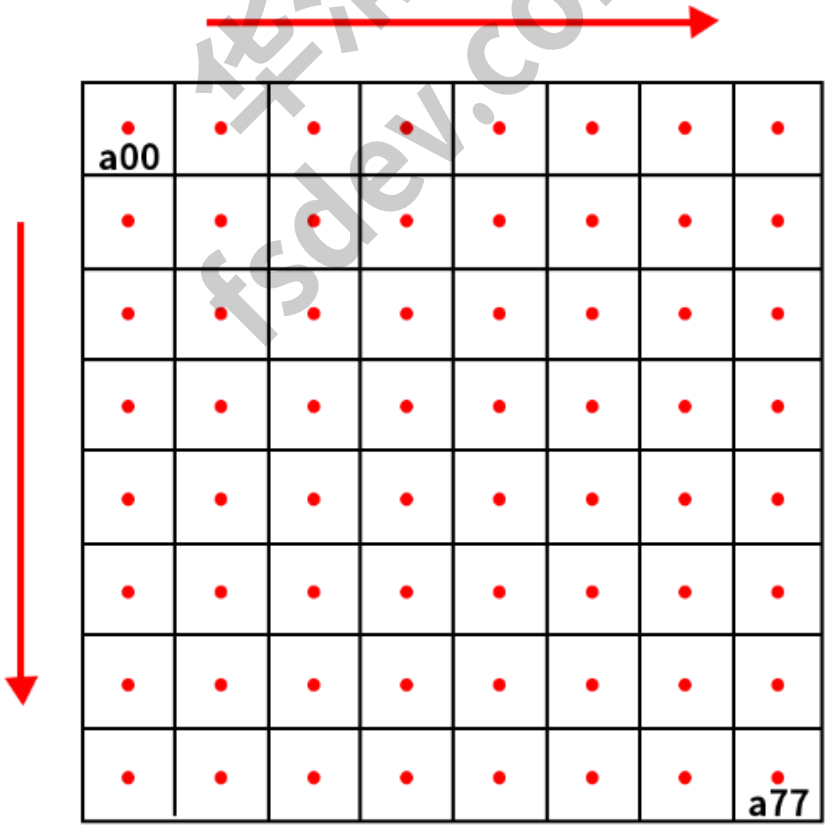
$$srcX = dstX * \frac{srcWidth}{dstWidth}$$

$$srcY = dstY * \frac{srcHeight}{dstHeight}$$

下面我们举例说明，假设原图像A大小为m*n，缩放后的目标图像B的大小为M*N。其中A的每一个像素点是已知的，B是未知的，我们想要求出目标图像B中每一个像素点（X,Y）的值，必须先找出像素（X,Y）在原图像A中对应的像素（x,y），再根据原图像A距离像素（x,y）最近的64个像素点作为计算目标图像B（X,Y）处像素值的参数，利用权重函数求出64个像素点的权重，图B像素（x,y）的值就等于64个像素点的加权叠加。

假如下图中的P点就是目标图像B在（X,Y）处根据上述公式计算出的对应于原图像A中的位置，P的坐标位置会出现小数部分，所以我们假设P点的坐标为（x+u,y+v），其中x、y表示整数部分，u、v表示小数部分，那么我们就可以得到其周围的最近的64个像素的位置，我们用a（i，j）（i，j=0,1,2,3,4,5,6,7）来表示，如下图所示。

华清远见 fsdev.com.cn



华清远见 HQYJ.COM

然后给出权重公式：

$$L(x)=\begin{cases}1 & x=0\\ \dfrac{a\sin\pi x\times\sin\dfrac{\pi x}{a}}{\pi^2x^2} & -a\le x\le a且x\ne 0\\ 0 & 其他\end{cases}$$

其中a通常取2或者3，当a=2时，该算法适用于图像缩小。a=3时，该算法适用于图像放大。

与双三次插值一样，这里也需要将像素点分行和列分别带入计算权重值，其他像素点也是这样的计算过程，最终我们就可以得到P所对应的目标图像B在（X,Y）处的像素值为：

$$S(x,y)=\sum_{i=[x]-a+1}^{[x]+a}\sum_{j=[y]-a+1}^{[y]+a}s_{ij}L(x-i)L(y-j)$$

其中$[x]$、$[y]$表示对坐标值向下取整，通过该方法就可以计算出新的图像中所有的像素点的像素值。

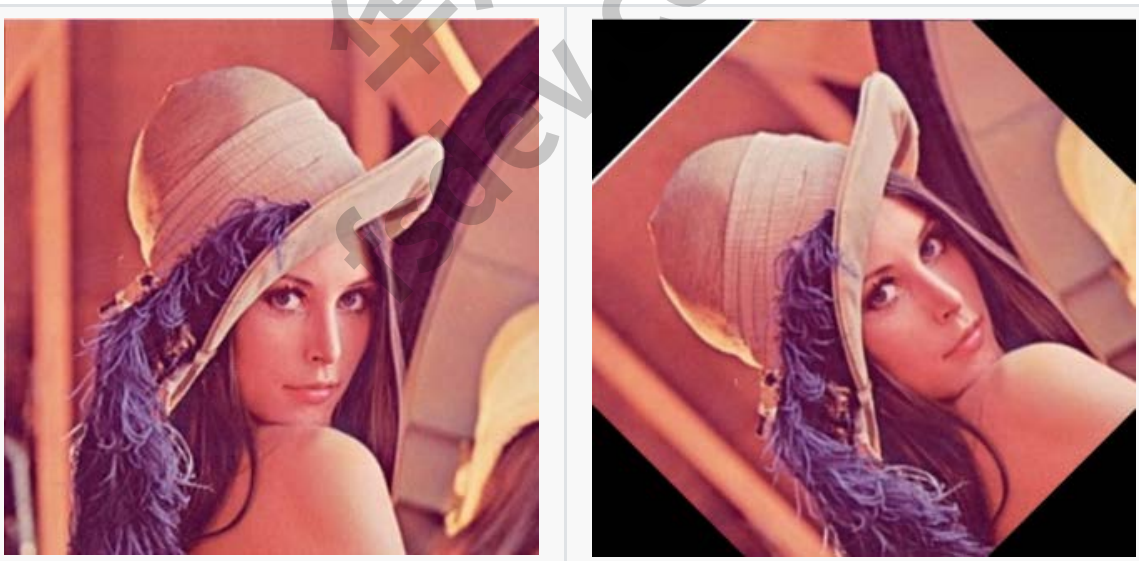
3.6 小结

最近邻插值的计算速度最快，但是可能会导致图像出现锯齿状边缘和失真，效果较差。双线性插值的计算速度慢一点，但效果有了大幅度的提高，适用于大多数场景。双三次插值、Lanczos插值的计算速度都很慢，但是效果都很好。

在OpenCV中，关于插值方法默认选择的都是双线性插值，且一般情况下双线性插值已经能满足大部分需求。

4. 边缘填充方式

为什么要填充边缘呢？我们已下图为例。



可以看到，左图在逆时针旋转45度之后原图的四个顶点在右图中已经看不到了，同时，右图的四个顶点区域其实是什么都没有的，因此我们需要对空出来的区域进行一个填充。右图就是对空出来的区域进行了像素值为（0，0，0）的填充，也就是黑色像素值的填充。除此之外，后续的一些图像处理方式也会用到边缘填充，这里介绍五个常用的边缘填充方法。

华清远见 fsdev.com.cn

华清远见 HQYJ.COM

4.1 边界复制（BORDER_REPLICATE）

边界复制会将边界处的像素值进行复制，然后作为边界填充的像素值，如下图所示，可以看到四周的像素值都一样。

1	1	1	1	1	2	3	4	4	4	4	4
1	1	1	1	1	2	3	4	4	4	4	4
1	1	1	1	1	2	3	4	4	4	4	4
1	1	1	1	1	2	3	4	4	4	4	4
1	1	1	1	1	2	3	4	4	4	4	4
5	5	5	5	5	6	7	8	8	8	8	8
9	9	9	9	9	10	11	12	12	12	12	12
13	13	13	13	13	14	15	16	16	16	16	16
13	13	13	13	13	14	15	16	16	16	16	16
13	13	13	13	13	14	15	16	16	16	16	16
13	13	13	13	13	14	15	16	16	16	16	16
13	13	13	13	13	14	15	16	16	16	16	16

4.2 边界反射（BORDER_REFLECT）

如下图所示，会根据原图的边缘进行反射。

华清远见 fsdev.com.cn

华清远见 HQYJ.COM

16	15	14	13	13	14	15	16	16	15	14	13
12	11	10	9	9	10	11	12	12	11	10	9
8	7	6	5	5	6	7	8	8	7	6	5
4	3	2	1	1	2	3	4	4	3	2	1
4	3	2	1	1	2	3	4	4	3	2	1
8	7	6	5	5	6	7	8	8	7	6	5
12	11	10	9	9	10	11	12	12	11	10	9
16	15	14	13	13	14	15	16	16	15	14	13
16	15	14	13	13	14	15	16	16	15	14	13
12	11	10	9	9	10	11	12	12	11	10	9
8	7	6	5	5	6	7	8	8	7	6	5
4	3	2	1	1	2	3	4	4	3	2	1

4.3 边界反射101（BORDER_REFLECT_101）

与边界反射不同的是，不再反射边缘的像素点，如下图所示。

16	15	14	13	14	15	16	15	14	13
12	11	10	9	10	11	12	11	10	9
8	7	6	5	6	7	8	7	6	5
4	3	2	1	2	3	4	3	2	1
8	7	6	5	6	7	8	7	6	5
12	11	10	9	10	11	12	11	10	9
16	15	14	13	14	15	16	15	14	13
12	11	10	9	10	11	12	11	10	9
8	7	6	5	6	7	8	7	6	5
4	3	2	1	2	3	4	3	2	1

华清远见 fsdev.com.cn

4.4 边界常数（BORDER_CONSTANT）

当选择边界常数时，还要指定常数值是多少，默认的填充常数值为0，如下图所示。

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	1	2	3	4	0	0
0	0	5	6	7	8	0	0
0	0	9	10	11	12	0	0
0	0	13	14	15	16	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

4.5 边界包裹（BORDER_WRAP）

如下图所示。

1	2	3	4	1	2	3	4	1	2	3	4
5	6	7	8	5	6	7	8	5	6	7	8
9	10	11	12	9	10	11	12	9	10	11	12
13	14	15	16	13	14	15	16	13	14	15	16
1	2	3	4	1	2	3	4	1	2	3	4
5	6	7	8	5	6	7	8	5	6	7	8
9	10	11	12	9	10	11	12	9	10	11	12
13	14	15	16	13	14	15	16	13	14	15	16
1	2	3	4	1	2	3	4	1	2	3	4
5	6	7	8	5	6	7	8	5	6	7	8
9	10	11	12	9	10	11	12	9	10	11	12
13	14	15	16	13	14	15	16	13	14	15	16

华清远见 fsdev.com.cn

5. 编程实例与步骤

需要通过元宇宙平台进入“人工智能虚拟仿真实验”界面和打开“华清远见人工智能虚拟仿真本地服务管理平台”软件。

5.1 （可选）使用“华清远见人工智能虚拟仿真本地服务管理平台”启动服务

服务启动一次之后就不需要再次启动了。

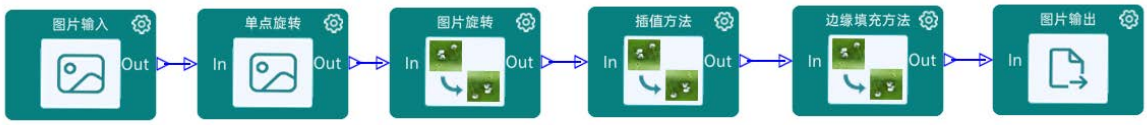
5.2 单击图片旋转实验进入

如果之前打开了进入了某个实验，可以点击“返回”回到主页即可。

华清远见 fsdev.com.cn

5.3 拖出组件并连线

根据实验原理，将我们的逻辑抽象成组件之后，连线如下图所示。



5.4 验证

点击验证，如显示校验成功，即代表逻辑无误，验证按钮与校验成功界面如下图所示。

华清远见 fsdev.com.cn

5.5 运行

点击运行，可以实现功能运行，并显示结果输出，如下图所示。

华清远见 HQYJ.COM

点击某张图片可放大查看效果。

5.6 代码生成

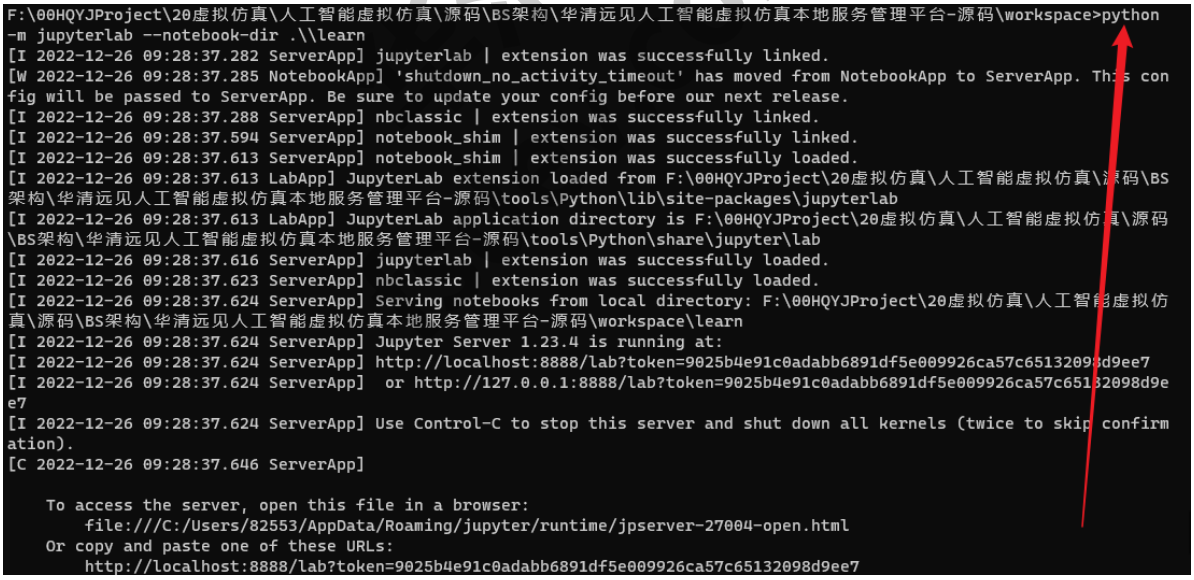
点击界面左下角的生成代码按钮，可生成本实验的Python代码，生成代码按钮如下图所示。

点击后可以查看该实验代码，代码会根据界面所选参数进行适配，点击复制可以将其复制，并粘贴在其它地方。

这里我们可以粘贴到软件的Jupyter中运行，“华清远见人工智能虚拟仿真本地服务管理平台”打开Jupyter，第一次打开Jupyter需要点击“打开CMD命令”，打开Jupyter后无需再次打开CMD，如下图所示。

华清远见 fsdev.com.cn

在弹出的CMD命令行中，输入python -m jupyterlab --notebook-dir .\\learn，点击回车，浏览器会自动跳转打开Jupyter，如下图所示。



```
F:\00HQYJProject\20虚拟仿真\人工智能虚拟仿真\源码\BS架构\华清远见人工智能虚拟仿真本地服务管理平台-源码\workspace>python
-m jupyterlab --notebook-dir .\\learn
[I 2022-12-26 09:28:37.282 ServerApp] jupyterlab | extension was successfully linked.
[W 2022-12-26 09:28:37.285 NotebookApp] 'shutdown_no_activity_timeout' has moved from NotebookApp to ServerApp. This con
fig will be passed to ServerApp. Be sure to update your config before our next release.
[I 2022-12-26 09:28:37.288 ServerApp] nbclassic | extension was successfully linked.
[I 2022-12-26 09:28:37.594 ServerApp] notebook_shim | extension was successfully linked.
[I 2022-12-26 09:28:37.613 ServerApp] notebook_shim | extension was successfully loaded.
[I 2022-12-26 09:28:37.613 LabApp] JupyterLab extension loaded from F:\00HQYJProject\20虚拟仿真\人工智能虚拟仿真\源码\BS
架构\华清远见人工智能虚拟仿真本地服务管理平台-源码\tools\Python\lib\site-packages\jupyterlab
[I 2022-12-26 09:28:37.613 LabApp] JupyterLab application directory is F:\00HQYJProject\20虚拟仿真\人工智能虚拟仿真\源码
\BS架构\华清远见人工智能虚拟仿真本地服务管理平台-源码\tools\Python\share\jupyter\lab
[I 2022-12-26 09:28:37.616 ServerApp] jupyterlab | extension was successfully loaded.
[I 2022-12-26 09:28:37.623 ServerApp] nbclassic | extension was successfully loaded.
[I 2022-12-26 09:28:37.624 ServerApp] Serving notebooks from local directory: F:\00HQYJProject\20虚拟仿真\人工智能虚拟仿
真\源码\BS架构\华清远见人工智能虚拟仿真本地服务管理平台-源码\workspace\learn
[I 2022-12-26 09:28:37.624 ServerApp] Jupyter Server 1.23.4 is running at:
[I 2022-12-26 09:28:37.624 ServerApp] http://localhost:8888/lab?token=9025b4e91c0adabb6891df5e009926ca57c65132098d9ee7
[I 2022-12-26 09:28:37.624 ServerApp]  or http://127.0.0.1:8888/lab?token=9025b4e91c0adabb6891df5e009926ca57c65132098d9e
e7
[I 2022-12-26 09:28:37.624 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirm
ation).
[C 2022-12-26 09:28:37.646 ServerApp]

    To access the server, open this file in a browser:
        file:///C:/Users/82553/AppData/Roaming/jupyter/runtime/jpserver-27004-open.html
    Or copy and paste one of these URLs:
        http://localhost:8888/lab?token=9025b4e91c0adabb6891df5e009926ca57c65132098d9ee7
```

如果没有自动跳转和打开浏览器，请输入下图中圈出的内容，每个人的该内容不一致，请复制自己的内容。

```
F:\00HQYJProject\20虚拟仿真\人工智能虚拟仿真\源码\BS架构\华清远见人工智能虚拟仿真本地服务管理平台-源码\workspace>python
-m jupyterlab --notebook-dir .\\learn
[I 2022-12-26 09:28:37.282 ServerApp] jupyterlab | extension was successfully linked.
[W 2022-12-26 09:28:37.285 NotebookApp] 'shutdown_no_activity_timeout' has moved from NotebookApp to ServerApp. This con
fig will be passed to ServerApp. Be sure to update your config before our next release.
[I 2022-12-26 09:28:37.288 ServerApp] nbclassic | extension was successfully linked.
[I 2022-12-26 09:28:37.594 ServerApp] notebook_shim | extension was successfully linked.
[I 2022-12-26 09:28:37.613 ServerApp] notebook_shim | extension was successfully loaded.
[I 2022-12-26 09:28:37.613 LabApp] JupyterLab extension loaded from F:\00HQYJProject\20虚拟仿真\人工智能虚拟仿真\源码\BS
架构\华清远见人工智能虚拟仿真本地服务管理平台-源码\tools\Python\lib\site-packages\jupyterlab
[I 2022-12-26 09:28:37.613 LabApp] JupyterLab application directory is F:\00HQYJProject\20虚拟仿真\人工智能虚拟仿真\源码
\BS架构\华清远见人工智能虚拟仿真本地服务管理平台-源码\tools\Python\share\jupyter\lab
[I 2022-12-26 09:28:37.616 ServerApp] jupyterlab | extension was successfully loaded.
[I 2022-12-26 09:28:37.623 ServerApp] nbclassic | extension was successfully loaded.
[I 2022-12-26 09:28:37.624 ServerApp] Serving notebooks from local directory: F:\00HQYJProject\20虚拟仿真\人工智能虚拟仿
真\源码\BS架构\华清远见人工智能虚拟仿真本地服务管理平台-源码\workspace\learn
[I 2022-12-26 09:28:37.624 ServerApp] Jupyter Server 1.23.4 is running at:
[I 2022-12-26 09:28:37.624 ServerApp] http://localhost:8888/lab?token=9025b4e91c0adabb6891df5e009926ca57c65132098d9ee7
[I 2022-12-26 09:28:37.624 ServerApp]  or http://127.0.0.1:8888/lab?token=9025b4e91c0adabb6891df5e009926ca57c65132098d9e
e7
[I 2022-12-26 09:28:37.624 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirm
ation).
[C 2022-12-26 09:28:37.646 ServerApp]

    To access the server, open this file in a browser:
        file:///C:/Users/82553/AppData/Roaming/jupyter/runtime/jpserver-27004-open.html
    Or copy and paste one of these URLs:
        http://localhost:8888/lab?token=9025b4e91c0adabb6891df5e009926ca57c65132098d9ee7
```

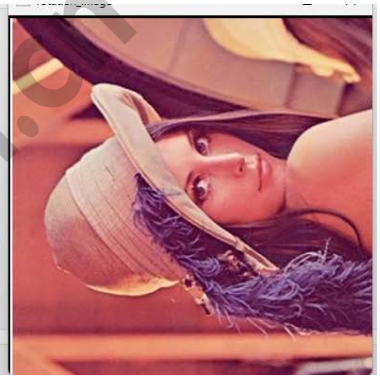
将代码输入到Jupyter中，注意这里需要往Jupyter导入一张demo.png，在大多数情况下，这张照片最好不要是其它后缀修改成png后缀而来，而是直接后缀就是png。点击运行可以查看结果。

6. 实验现象

可以看到，图片旋转组件中不同的参数对于图片的旋转及缩放后的结果不同，实验结果为：

```python
import cv2

if __name__ == "__main__":
    path = "./images/lena.png"
    image_np = cv2.imread(path)
    img_shape = image_np.shape
    # 对图片进行旋转
    M = cv2.getRotationMatrix2D((img_shape[0] / 2, img_shape[1] / 2), 90, 1)
    rotation_image = cv2.warpAffine(image_np, M, (img_shape[0], img_shape[1]))
    cv2.imshow("rotation_image", rotation_image)
    cv2.waitKey(0)
```



华清远见 fsdev.com.cn